Backend Engineer Intern Assignment
Build a Simple Doctor–Patient Consultation Backend
Duration
1–2 days

Objective
Build the backend for a simple healthcare consultation platform where patients can:

- Register and log in
- View available doctors
- Start a consultation
- Exchange chat messages with the doctor
- End the consultation

This assignment is designed to evaluate your understanding of backend fundamentals, API

design, database modeling, authentication, and clean coding practices.

Tech Stack

Required

- Node.js
- Express.js
- MySQL or PostgreSQL
- JWT Authentication
- Prisma / Sequelize / TypeORM (or raw SQL)
- Git

Functional Requirements

1. Authentication

Implement JWT-based authentication.

Users can register as:

- Patient
- Doctor

User Fields

- Name
- Email
- Password
- Role

APIs

POST /auth/register

POST /auth/login

GET /profile

Requirements

- Passwords must be securely hashed.
- Only authenticated users can access protected APIs.

---

2. Doctors

Each doctor should have:

- Name
- Specialization
- Years of Experience

APIs

GET /doctors

GET /doctors/:id

Patients should be able to browse the list of available doctors.

---

3. Consultation

A patient can initiate a consultation with a

doctor.

Each consultation should contain:

- Patient
- Doctor
- Status
- Created At

Consultation Status

- Pending
- Active
- Completed

APIs

POST /consultations

GET /consultations

GET /consultations/:id

PATCH /consultations/:id/status

Rules

Only patients can create consultations.
Only the assigned doctor can change the consultation status.
Completed consultations cannot be modified.

---

4. Chat

Patients and doctors should be able to exchange messages within an active consultation.
Each message should contain:

Consultation ID
Sender
Message
Timestamp

APIs

POST /consultations/:id/messages

GET /consultations/:id/messages

Rules

- Only the assigned doctor and patient can send messages.
- Messages should be returned in chronological order.
- Messages cannot be sent once the consultation is completed.

Database Design

Design appropriate tables for:

- Users
- Doctors (or doctor profile)
- Consultations
- Messages

Use proper relationships between the tables.

Validation & Error Handling

Implement basic validation:

- Valid email format
- Required fields
- Duplicate email registration
- Invalid consultation IDs
- Unauthorized access
- Invalid consultation state changes

Return appropriate HTTP status codes and meaningful error messages.

Project Structure

Use a clean and organized folder structure. For

example:

```
src/
├── controllers/
├── routes/
├── services/
├── models/
├── middleware/
├── config/
├── utils/
└── app.js
```

Feel free to organize your project differently if it remains clean and maintainable.

API Documentation

Provide either:

- A Postman Collection, or
- A README with all API endpoints and

sample requests.

Bonus (Optional)

If time permits, you may implement any of the following:

- Real-time chat using Socket.io
- Pagination for consultation history
- Swagger API documentation
- Docker setup

These are completely optional and will only be considered as bonus points.

Submission

Share:

GitHub Repository

README containing:

Setup instructions

Database schema (ER diagram or screenshots are acceptable)

API documentation

Any assumptions made during development

Evaluation Criteria

Criteria	Weight
Database Design	20%
REST API Design	25%
Authentication & Authorization	20%

Business Logic	20%
Code Quality & Project Structure	10%
Validation & Error Handling	5%

What We're Looking For

We're not expecting a production-ready application. Instead, we want to understand how you approach backend development, structure your code, design APIs, and implement business logic. A clean, well-organized solution that correctly implements the core features is preferred over a larger but incomplete implementation. Document any assumptions you make in your README.